Indexing Pangenomes: Suffix Arrays vs. de Bruijn Graphs, Plain and Wheeler

Brady Bock Bob Dong Andy Nguyen Albert Wang

EN.601.447 Project Proposal (revised)

1 Introduction

A pangenome is a collection of genomes from one species that are highly similar to one another. We study two questions about such a collection:

Q1. Genome comparison. How similar is each pair of genomes, and where along the genomes is sequence shared or different?

Q2. Read placement. Given a read, which genome is it most similar to a substring of, and at which position?

Both questions can be answered from an index of the collection. The choice of index determines what the answers cost to build, store, and compute, and how exact they are.

A string index such as a suffix array [MM93] or FM-index [FM00] stores every genome in full. It answers both questions exactly, down to the base and the position, but pays in space for the redundancy. A de Bruijn graph stores each distinct k-mer once, so shared sequence collapses and the structure can be far smaller. Labeling each k-mer with the set of genomes that contain it (its *colors* [ICT+12]) lets the graph report which genomes contain a sequence. The price is twofold. The graph forgets positions, and it contains paths that switch between genomes at shared k-mers, or run through repeats within one genome, spelling sequences that occur in no input. Such paths can place a read in a genome that does not contain it. A recent benchmark of de Bruijn graph pangenome tools measures time, memory, and mapping rate, but leaves accuracy evaluation out of scope [BŞZ+26].

We compare the two families directly on Q1 and Q2. For each index we measure the cost of building it (time and peak memory), of storing it (size), and of answering each question (time per query), and we score its answers against known ground truth. On the string side we build the suffix array three ways: prefix doubling [MM93], DC3 [KS03], and SA-IS induced sorting [NZC11]. From it we derive the LCP array [KLA+01], the BWT, and an FM-index. On the graph side we store a colored de Bruijn graph both as an explicit hash-based adjacency structure and in succinct Wheeler form [GMS17, BOSS12]. We build the Wheeler form from the suffix array of the reversed collection: suffixes that share their first k characters form one node, already in Wheeler order, and the node's out-edge labels are the distinct BWT characters in that block. The three construction algorithms therefore serve both families.

2 Research goals

Queries. We define both questions precisely, so that every index answers the same thing.

- **Q1 (genome comparison)** returns (a) a pairwise similarity matrix and (b) for each genome, the intervals it shares with the others and the positions where it differs.

  - *String side.* The suffix array, LCP array, and a document array (recording the genome each suffix comes from) give, for every position, the longest match shared with another genome. The point where shared matches stop marks a difference at base resolution.
  - *Graph side.* Colors give the shared k-mer content of every pair, which we convert to a divergence estimate as in Mash [OTM+16]. Bubbles whose branches carry different colors mark differences [ICT+12].

- **Q2 (read placement)** returns the genomes, and on the string side the offsets, that best explain a read.

  - *Exact mode:* the read must occur in full. The string side locates it and maps each hit to its genome. The graph searches for a path spelling the read and intersects the colors along that path.
  - *Voting mode,* which tolerates sequencing errors: each k-mer of the read votes for the genomes that contain it, and the genomes with the most votes win.

Both families see identical k-mer memberships, so in voting mode and for k-mer similarity they differ only in cost. They differ in accuracy in three places, which we measure: spurious whole-read matches on the graph, the absence of positions on the graph, and the resolution at which differences are located, since variants fewer than k bases apart merge into a single bubble.

Methods. Graph edges come only from $(k+1)$-mers observed in some genome, so the explicit and Wheeler graphs accept exactly the same paths. Colors are stored as identifiers of distinct color sets, because most k-mers share one of a few sets. The FM-index samples the suffix array at a fixed rate, which we report. Index construction and queries are implemented in C++, so that timings reflect the algorithms rather than interpreter overhead. Python only drives experiments and plots.

Evaluation. Table 1 lists what we measure. We vary the number of genomes, their divergence, k, read length, and sequencing error rate. We also test three predictions:

(i) the suffix array and FM-index grow with the total length n, while the graph grows with the number of distinct variants, roughly $N + vk$ nodes for a genome of length N carrying v variants in total, and the number of BWT runs r [GNP20] grows with v as well;

(ii) the graph accepts a read that joins alleles from two genomes whenever the two switched variants are at least k bases apart, so false placements should fall as k grows;

(iii) most reads cannot be placed in a single genome: with per-genome variant rate d and read length ℓ, only about $1 - (1-d)^{\ell}$ of reads cover a variant unique to their source, about 14% for $d = 0.1\%$ and $\ell = 150$.

We check correctness against brute-force search on small inputs. At full scale we use differential tests: the three suffix arrays must be identical, the explicit and Wheeler graphs must answer every query identically, and every substring of length at least k of any input must be accepted.

Aspect	Measured for every index
Build	Construction time and peak memory, for each suffix array algorithm and each graph form.
Size	Bytes per input base for the suffix array, FM-index, both graph forms, and the colors; the BWT run count r against n and graph size.
Q1	Time per genome pair; precision and recall of located differences against the variant log; error of the estimated divergence.
Q2	Time per read and per reported hit; whether the true source is reported; number of genomes reported against the ideal set (all genomes containing the read); false placements; position accuracy (string side).

Table 1: Evaluation metrics. Ground truth comes from the simulator's variant log and from the known source of every simulated read.

Input data. *Synthetic.* We start from the *E. coli* K-12 MG1655 reference genome [BPB+97] and simulate collections of near-identical copies. Each copy is mutated independently from the reference with SNPs and short indels, and the simulator logs every variant, so we know the ground truth. We vary the number of copies and the divergence between them. As a non-repetitive control we use the same number of uniformly random genomes of the same length. Reads are sampled from the copies with their source genome and offset recorded, optionally with sequencing errors. Query patterns are drawn from the copies, from mutated versions of those samples, and from random sequence. We also build recombinant patterns that join alleles from different copies, since these are the ones the graph is most likely to match falsely. *E. coli* contains repeats longer than k, so we also index a single copy to separate repeat-induced errors from recombination-induced ones. All data is regenerated from fixed seeds by a single command.

Real. We will also evaluate on two real collections: (1) a large set of SARS-CoV-2 genomes, which at about 30 kb each lets us scale to thousands of genomes; and (2) a smaller set of complete *E. coli* or *Salmonella* assemblies, about 5 Mb each, whose differences include gene gain and loss as well as point mutations. Real genomes need preprocessing: ambiguous bases (N), reverse-complement orientation, and the arbitrary start position of circular chromosomes. Real data has no variant log. For Q2, ground truth comes from reads sampled from the real genomes. For Q1, the exact string-side answers serve as ground truth for the graph's answers.

Milestones.

1. **Foundations.** Genome and read simulators; download and preprocessing of the real collections; brute-force oracle; explicit colored de Bruijn graph.
2. **All indexes working.** Prefix doubling, DC3, and SA-IS; LCP array, BWT, FM-index, and document array; the Wheeler-form graph built from the reversed suffix array, with colors. Differential tests pass.
3. **Queries and pipeline.** Q1 and Q2 on every index, and a single command-line script that runs every experiment and regenerates all figures.
4. **Real data, analysis, and write-up.**

Stretch goals.

1. **Positions on the graph.** Sample (genome, offset) pairs on Wheeler graph nodes so that the graph can report where a read occurs, and measure how much of its space advantage remains.

2. **Run-length FM-index.** Build an FM-index whose size is proportional to the number of BWT runs r [GNP20], and test whether the graph's space advantage survives once the string side also exploits repetitiveness.
3. **Harder data.** Larger and more divergent real collections, and a simulator mode in which copies inherit variants along a tree, as in real populations.
4. **Visualization.** A script that draws a small Wheeler de Bruijn graph and the search interval shrinking step by step.

Tentative roles.

1. Prefix doubling, DC3, the LCP array, and the FM-index with its document array.
2. SA-IS and the Wheeler-form de Bruijn graph, including its construction from the reversed suffix array.
3. The explicit colored de Bruijn graph, colors for both graph forms, and the Q1 and Q2 query layer.
4. Synthetic and real data, read simulation, the brute-force oracle and differential tests, and the reproduction script.

References

[BOSS12] Alexander Bowe, Taku Onodera, Kunihiko Sadakane, and Tetsuo Shibuya. Succinct de Bruijn graphs. In *Algorithms in Bioinformatics (WABI 2012)*, volume 7534 of *Lecture Notes in Computer Science*, pages 225–235. Springer, 2012.

[BPB+97] Frederick R. Blattner, Guy Plunkett, Craig A. Bloch, Nicole T. Perna, Valerie Burland, Monica Riley, Julio Collado-Vides, Jeremy D. Glasner, Christopher K. Rode, George F. Mayhew, Jason Gregor, Nelson Wayne Davis, Heather A. Kirkpatrick, Michael A. Goeden, Debra J. Rose, Bob Mau, and Ying Shao. The complete genome sequence of *Escherichia coli* K-12. *Science*, 277(5331):1453–1462, 1997.

[BŞZ+26] Zülal Bingöl, Berkan Şahin, Klea Zambaku, Ricardo Roman-Brenes, Konstantina Koliogeorgi, Can Firtina, Onur Mutlu, and Can Alkan. De Bruijn graphs for pangenomics: In-depth performance benchmarking of de Bruijn graph-based tools for read mapping. *Briefings in Bioinformatics*, 27(4):bbag440, 2026.

[FM00] Paolo Ferragina and Giovanni Manzini. Opportunistic data structures with applications. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 390–398. IEEE, 2000.

[GMS17] Travis Gagie, Giovanni Manzini, and Jouni Sirén. Wheeler graphs: A framework for BWT-based data structures. *Theoretical Computer Science*, 698:67–78, 2017. Algorithms, Strings and Theoretical Approaches in the Big Data Era (In Honor of the 60th Birthday of Professor Raffaele Giancarlo).

[GNP20] Travis Gagie, Gonzalo Navarro, and Nicola Prezza. Fully functional suffix trees and optimal text searching in BWT-runs bounded space. *Journal of the ACM*, 67(1), January 2020.

[ICT+12] Zamin Iqbal, Mario Caccamo, Isaac Turner, Paul Flicek, and Gil McVean. De novo assembly and genotyping of variants using colored de Bruijn graphs. *Nature Genetics*, 44(2):226–232, 2012.

[KLA+01] Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park. Linear-time longest-common-prefix computation in suffix arrays and its applications. In *Combinatorial Pattern Matching (CPM 2001)*, volume 2089 of *Lecture Notes in Computer Science*, pages 181–192. Springer, 2001.

[KS03] Juha Kärkkäinen and Peter Sanders. Simple linear work suffix array construction. In Jos C. M. Baeten, Jan Karel Lenstra, Joachim Parrow, and Gerhard J. Woeginger, editors, *Automata, Languages and Programming*, pages 943–955, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[MM93] Udi Manber and Gene Myers. Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5):935–948, 1993.

[NZC11] Ge Nong, Sen Zhang, and Wai Hong Chan. Two efficient algorithms for linear time suffix array construction. *IEEE Transactions on Computers*, 60(10):1471–1484, 2011.

[OTM+16] Brian D. Ondov, Todd J. Treangen, Páll Melsted, Adam B. Mallonee, Nicholas H. Bergman, Sergey Koren, and Adam M. Phillippy. Mash: Fast genome and metagenome distance estimation using MinHash. *Genome Biology*, 17:132, 2016.